MODEL-RECOMMENDATION.md — Which Claude Model to Use

> This document explains the difference between Claude Sonnet and Claude Opus in the context of the Muralis project, and gives a clear recommendation for each type of task.

The short answer

Use Sonnet 4.5 for most coding tasks. Use Opus 4 only for architecture, debugging, and cross-file reasoning.

The longer answer requires understanding what the models are actually good at and where they differ for a project like this.

What is the actual difference?

Think of Claude Sonnet as a very fast, very capable senior developer who has read your spec files and can generate correct code quickly. Think of Claude Opus as a principal engineer who thinks slower but catches subtle problems that Sonnet might miss.

For most of the steps in `DEV-PLAN.md`, Sonnet is the right tool. The spec files have already done the hard thinking — they contain the exact type names, field names, method signatures, and behavioural contracts. Sonnet reads those specs and generates code that matches them. It is fast, it follows explicit instructions well, and it will correctly implement a `TreeMap<Long, Long>` with `Comparator.reverseOrder()` when the spec tells it to.

Opus earns its cost when the task requires holding many constraints in mind simultaneously, reasoning about subtle interactions between components, or diagnosing a bug that spans multiple files. These moments exist in this project, but they are not every task.

Task-by-task recommendation

Use Sonnet 4.5 for:

All code generation steps (Steps 0–8 in DEV-PLAN.md) The specs are precise enough that Sonnet will follow them correctly. Sonnet is significantly faster, which matters when you are iterating on a code generation prompt multiple times.

Updating a single file "Add a new instrument spec to DATA-CONTRACTS.md" or "change the bubble decay minimum from 1s to 2s in SPEC-rendering.md" — these are single-file changes that Sonnet handles without effort.

Writing unit tests The test cases in Step 8 are derived directly from invariants listed in the spec files. Sonnet reads the invariant checklist and writes tests that verify each item. This is mechanical work that does not require deep cross-file reasoning.

Fixing a specific compilation error "This class has a missing import" or "this method signature doesn't match the interface" — paste the error, paste the relevant spec section, Sonnet fixes it.

Generating boilerplate `build.gradle.kts`, `logback.xml`, the `META-INF/services` file — these are well-defined outputs with no ambiguity.

Use Opus 4 for:

Diagnosing a bug that spans multiple files If your order book shows wrong prices after a reconnect, the bug could be in `BinanceAdapter` (bootstrap sequence), `OrderBookEngine` (applySnapshot/applyDelta), or `BinanceMessageParser` (parsing). Opus is better at holding the entire pipeline in mind and identifying where the invariant is violated.

Reviewing generated code for architectural violations After Sonnet generates a package, you can ask Opus to review it against the invariant checklists across all spec files simultaneously. Opus is more likely to catch a subtle violation like "this class imports from `com.muralis.ingestion`" buried in a file that otherwise looks correct.

Designing anything new If you want to add footprint charts (Phase 2), do not ask Sonnet to design the data model. Bring that conversation to Opus. The design decisions — what types to introduce, what invariants to add to DATA-CONTRACTS.md, how the engine thread model changes — require the kind of holistic reasoning that Opus handles better.

The bootstrap sequence logic The Binance snapshot bootstrap sequence (Step 5 in `SPEC-ingestion.md`) is the most complex piece of logic in the entire Phase 1 codebase. If it does not work correctly, the order book will silently have wrong state. If you suspect a bootstrap bug, use Opus to diagnose it — it will reason through the sequence ID validation logic more carefully.

Any prompt that requires reading more than 3 spec files simultaneously A prompt like "does the way OrderBookEngine builds the RenderSnapshot match what LadderPainter expects, given the price scale defined in DATA-CONTRACTS.md?" spans four files. Opus handles this better.

Practical workflow

```
For each step in DEV-PLAN.md:
```

```
1. Open Claude Code with Sonnet 4.5
2. Run the prompt exactly as written in DEV-PLAN.md
3. Review the generated code briefly — does it look reasonable?
4. Run ./gradlew build
5. Perform the manual verification described in the step
6. If something is wrong and the fix is obvious → fix it with Sonnet
7. If something is wrong and you cannot identify why → switch to Opus,
   paste the relevant spec sections, describe the symptom, ask for diagnosis
```

Sonnet for generation. Opus for diagnosis. This combination keeps costs reasonable while getting the best reasoning where it matters.

A note on context window and spec files

One practical advantage of having all your decisions in spec files is that both models can access them equally. The spec files are plain text in your project directory — Claude Code reads them at the start of each session. This means you are not relying on either model's training knowledge about Binance's API or JavaFX's Canvas performance characteristics — the answers to those questions are already written down in the specs. Both models perform better when the answer they need is explicitly in front of them rather than recalled from training.

This is the core benefit of the SDD approach: it levels up Sonnet to behave more like Opus for well-specified tasks, because the specification does the architectural reasoning in advance.

Cost perspective

Opus is significantly more expensive per token than Sonnet. For a solo developer building a personal tool, this matters. The nine steps in `DEV-PLAN.md` are structured to use Sonnet for the majority of generation work. If you use Opus only for the three or four moments listed above (cross-file diagnosis, Phase 2 design), the cost difference is manageable and the quality benefit is real.

The worst pattern — and the one to avoid — is using Opus for every generation task and Sonnet for nothing. That is expensive without being meaningfully better for well-specified code generation. The specs exist precisely to make Sonnet reliable for those tasks.

Final recommendation summary

Task	Model	Reason
Steps 0–8 code generation	**Sonnet 4.5**	Specs are precise enough
Fixing compilation errors	**Sonnet 4.5**	Single-file, well-defined
Writing unit tests	**Sonnet 4.5**	Derived directly from checklists
Cross-file bug diagnosis	**Opus 4**	Requires holistic reasoning
Phase 2 design decisions	**Opus 4**	New constraints, no spec yet
Architecture review	**Opus 4**	Multi-file invariant checking
Bootstrap sequence debugging	**Opus 4**	Most complex logic in Phase 1

Start with Sonnet. Switch to Opus when you are stuck or designing something new. Switch back to Sonnet when you have a spec to implement.

MODEL-RECOMMENDATION.md v1.0